

---

# Laboratório 03

## Primeiro Contacto com Containers

Infraestruturas de Computação na Nuvem | Aula 03

---

### Objetivo e enquadramento

---

A Aula 03 estabelece, no plano teórico, a distinção entre virtualização completa, na qual o hypervisor apresenta hardware virtual e cada instância executa um núcleo próprio, e virtualização ao nível do sistema operativo, na qual as instâncias partilham o núcleo do anfitrião. Este laboratório destina-se a tornar essa distinção observável em equipamento próprio.

O trabalho decorre no computador pessoal de cada aluno e não requer acesso ao servidor da unidade curricular. Destina-se a exploração durante a aula, não constituindo trabalho a entregar. Não se pressupõe experiência prévia com containers.

O laboratório utiliza Docker por ser a implementação de containers de instalação mais simples em qualquer sistema operativo. Os mecanismos de núcleo que a sustentam, designadamente *namespaces* e *cgroups*, constituem matéria da Aula 04. O objetivo neste momento não é dominar a ferramenta, mas observar o comportamento que a teoria descreve.

### Resultados de aprendizagem

---

No final deste laboratório, o aluno deve ser capaz de:

- Executar e inspecionar um container, distinguindo imagem de instância em execução.
- Demonstrar experimentalmente a partilha do núcleo entre container e anfitrião.
- Identificar, na sua própria máquina, onde se situa o hypervisor da Aula 03.
- Publicar um serviço em rede a partir de um container e justificar o mapeamento de portos.
- Construir uma imagem a partir de um **Dockerfile** e explicar a sua estrutura em camadas.
- Aplicar limites de recursos a um container e observar o seu efeito.

### Como utilizar este guião

---

O guião não constitui trabalho a entregar. Destina-se a ser percorrido durante a aula, ao ritmo de cada aluno, com acompanhamento da docente.

Ao longo do texto surgem caixas *Pare e observe*. Assinalam os momentos em que interessa deter a execução e interpretar o resultado obtido, isoladamente ou em discussão com os colegas, antes de prosseguir. São esses momentos, e não a execução dos comandos, que constituem o objeto do trabalho.

## Percurso

---

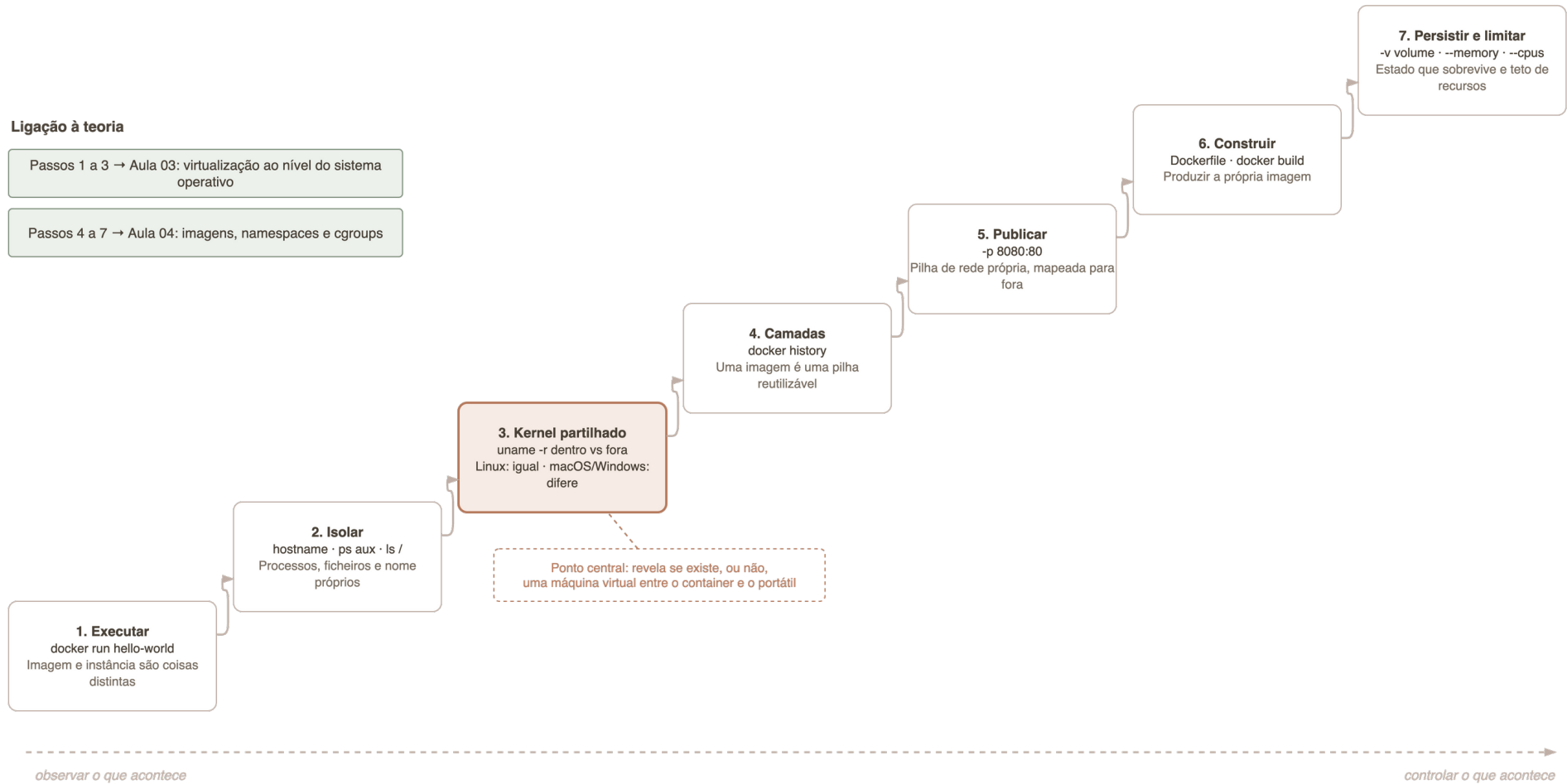
A figura da página seguinte apresenta a sequência do guião. Os passos progridem da simples execução de um comando até ao controlo dos recursos atribuídos, e o passo 3 assinala a observação central, aquela que revela a arquitetura de virtualização do próprio computador.

## Percurso do laboratório: do primeiro comando ao controlo de recursos

### Ligação à teoria

Passos 1 a 3 → Aula 03: virtualização ao nível do sistema operativo

Passos 4 a 7 → Aula 04: imagens, namespaces e cgroups



# 1 Instalação

---

## 1.1 Windows e macOS

Instale o **Docker Desktop** a partir de <https://docs.docker.com/get-started/get-docker/>. Aceite as opções apresentadas por omissão. Em Windows, a instalação requer que o WSL 2 se encontre ativo; o instalador deteta a sua ausência e indica o procedimento.

Concluída a instalação, inicie a aplicação Docker Desktop e aguarde que o indicador de estado apresente **Engine running**. Os comandos deste guião são executados em seguida no terminal do sistema, PowerShell em Windows ou Terminal em macOS, e não dentro da aplicação.

## 1.2 Linux

Instale o **Docker Engine** através do repositório oficial da distribuição, seguindo <https://docs.docker.com/engine/install/>. Após a instalação, acrescente o utilizador ao grupo **docker** para dispensar `sudo`:

```
sudo usermod -aG docker $USER
```

Termine a sessão e volte a iniciá-la para que a alteração produza efeito.

## 1.3 Verificação

```
docker --version  
docker info
```

O primeiro comando indica a versão instalada. O segundo apresenta o estado do motor e confirma a ligação ao serviço.

### Confirme antes de avançar

`docker --version` devolve uma linha com a versão, e `docker info` termina sem mensagem de erro. Caso surja **Cannot connect to the Docker daemon**, o serviço não se encontra em execução: consulte a secção de resolução de problemas.

# 2 O primeiro container

---

Execute:

```
docker run hello-world
```

### Saída esperada

```
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
...  
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

A primeira linha merece atenção. A imagem não existia localmente, pelo que foi transferida de um repositório remoto antes da execução. Esta separação entre **imagem**, o conjunto de ficheiros imutável que serve de modelo, e **container**, a instância em execução criada a partir

dela, é análoga à distinção entre um ficheiro de disco virtual e uma máquina virtual em execução, discutida na Aula 03.

Confirme que o container terminou mas continua registado:

```
docker ps
docker ps -a
```

O primeiro comando lista apenas containers em execução e não apresentará resultados. O segundo inclui os já terminados, entre os quais o **hello-world** com estado **Exited**.

Um container termina quando o processo que lhe deu origem termina. Não existe, ao contrário de uma máquina virtual, um sistema de arranque a manter a instância ativa na ausência de trabalho.

### 3 Observar o isolamento

---

Inicie um container interativo baseado em Alpine Linux, uma distribuição de dimensão reduzida:

```
docker run -it --rm alpine sh
```

A opção **-it** liga o terminal ao container e **-rm** remove-o automaticamente ao terminar. A linha de comandos passa a ser a do interior do container.

Execute, já dentro do container:

```
hostname
ps aux
ls /
cat /etc/os-release
```

Observe cada resultado. O **hostname** não corresponde ao da máquina. A lista de processos contém dois ou três elementos, e o processo **sh** apresenta o identificador 1, posição reservada ao primeiro processo de um sistema. A raiz do sistema de ficheiros não contém os ficheiros pessoais. O ficheiro **/etc/os-release** identifica Alpine Linux, independentemente do sistema operativo instalado no computador.

Pare e observe

**Observação 1.** Observe o resultado de **ps aux** dentro do container. Quantos processos são visíveis? Compare com o número de processos em execução no seu computador. Justifique a diferença em termos do isolamento descrito na Aula 03.

**Não termine ainda este container.** A secção seguinte utiliza-o.

### 4 A prova da partilha do núcleo

---

Esta é a observação central do laboratório e liga-se diretamente à distinção estabelecida na aula.

Ainda dentro do container, execute:

```
uname -r
```

Registre a versão apresentada. Termine o container com `exit` e execute, agora no seu computador:

```
uname -r
```

#### Atenção

Em Windows, `uname` não existe. Utilize antes, em PowerShell, o comando `wsl uname -r`, que consulta o núcleo Linux do subsistema WSL 2.

### 4.1 Interpretação do resultado

O resultado depende do sistema operativo do computador, e a diferença é ela própria matéria da Aula 03.

**Em Linux.** As duas versões coincidem exatamente. O container não possui núcleo próprio: executa sobre o núcleo do anfitrião, que apenas lhe apresenta uma vista isolada dos recursos. Alpine e a distribuição instalada partilham o mesmo núcleo e diferem somente no espaço de utilizador.

**Em Windows ou macOS.** As duas versões diferem, e o núcleo comunicado dentro do container é um núcleo Linux, embora o sistema operativo instalado não o seja. A explicação encontra-se na aula: o Docker Desktop executa uma máquina virtual Linux sobre um hypervisor do sistema, o Hyper-V ou WSL 2 em Windows, o Virtualization.framework em macOS. O container partilha o núcleo dessa máquina virtual, e não o do computador.

Daqui decorre uma observação de alcance mais geral. Em Windows e macOS coexistem, na mesma máquina, os dois modelos de virtualização estudados na aula: virtualização completa, que sustenta a máquina virtual Linux, e virtualização ao nível do sistema operativo, que produz o container no seu interior. A afirmação corrente de que containers dispensam máquinas virtuais é, nestes sistemas, incorreta.

#### Pare e observe

**Observação 2.** Compare as duas versões de núcleo obtidas, tendo presente o seu sistema operativo. Diga se coincidem e justifique. Caso não coincidam, indique onde se situa o hypervisor e que papel desempenha.

## 5 Imagens e camadas

Transfira a imagem do servidor web Nginx sem a executar:

```
docker pull nginx:alpine
docker images
```

A coluna `SIZE` indica a dimensão da imagem. Compare-a com a de uma máquina virtual equivalente, tipicamente alguns gigabytes, e note a diferença de ordem de grandeza: a imagem não contém núcleo nem controladores de dispositivo, apenas o espaço de utilizador necessário à aplicação.

Inspecione a estrutura interna:

```
docker history nginx:alpine
```

Cada linha corresponde a uma **camada**, produzida por uma instrução de construção. As camadas são imutáveis e partilhadas entre imagens que delas derivem, o que evita a duplicação de conteúdo comum. O mecanismo é análogo ao dos discos virtuais diferenciais discutidos na Aula 03, nos quais um disco base é partilhado por várias instâncias.

#### Pare e observe

**Observação 3.** Repare na dimensão de `nginx:alpine` e o número de camadas apresentado por `docker history`. Explique por que motivo duas imagens derivadas da mesma base ocupam, em conjunto, menos espaço do que a soma das suas dimensões individuais.

## 6 Publicar um serviço

Inicie o servidor web em segundo plano, mapeando o porto 8080 do computador para o porto 80 do container:

```
docker run -d --name web -p 8080:80 nginx:alpine
docker ps
```

Abra `http://localhost:8080` num navegador. Deverá surgir a página de boas-vindas do Nginx.

O mapeamento é necessário porque o container possui uma pilha de rede própria, com endereço distinto do da máquina. Sem `-p`, o serviço permanece acessível apenas no interior da rede de containers. A opção estabelece uma regra de reencaminhamento entre o porto do anfitrião e o porto do container.

Consulte os registos e o interior do container em execução:

```
docker logs web
docker exec -it web sh
```

Dentro do container, confirme que o servidor está ativo e observe os ficheiros servidos:

```
ls /usr/share/nginx/html
exit
```

#### Confirme antes de avançar

A página responde em `localhost:8080` e `docker logs web` apresenta ao menos um pedido HTTP registado, correspondente ao acesso efetuado pelo navegador.

Termine e remova o container:

```
docker stop web
docker rm web
```

## 7 Construir uma imagem

Crie um diretório de trabalho e, no seu interior, dois ficheiros.

`index.html`:

```
<!DOCTYPE html>
<html lang="pt">
<head><meta charset="utf-8"><title>ICN</title></head>
<body>
  <h1>Infraestruturas de Computacao na Nuvem</h1>
  <p>Pagina servida a partir de um container construido localmente.</p>
</body>
</html>
```

Dockerfile, sem extensão:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

O ficheiro declara três elementos: a imagem base sobre a qual se constrói, o ficheiro a copiar para o seu interior, e o porto que a aplicação utiliza. A instrução **EXPOSE** tem valor documental e não substitui o mapeamento **-p** na execução.

Construa e execute:

```
docker build -t icn-site:v1 .
docker run -d --name meusite -p 8080:80 icn-site:v1
```

Aceda novamente a <http://localhost:8080>. A página apresentada é agora a construída localmente.

Pare e observe

**Observação 4.** Confirme que a página servida é a sua. Execute `docker images` e compare a dimensão de `icn-site:v1` com a de `nginx:alpine`. Explique a diferença observada com base no modelo de camadas.

## 7.1 Reconstrução e reutilização de camadas

Altere o texto de `index.html` e reconstrua:

```
docker build -t icn-site:v2 .
```

Observe as mensagens de construção. As camadas anteriores à alteração são reaproveitadas, assinaladas por **CACHED**, e apenas a camada correspondente ao ficheiro alterado é reconstruída. Este comportamento explica a recomendação corrente de colocar num **Dockerfile** as instruções menos voláteis antes das mais voláteis.

## 8 Persistência

A camada de escrita de um container é descartada com ele. Confirme experimentalmente:

```
docker exec meusite sh -c "echo teste > /tmp/ficheiro.txt"
docker exec meusite cat /tmp/ficheiro.txt
docker stop meusite && docker rm meusite
docker run -d --name meusite -p 8080:80 icn-site:v1
docker exec meusite cat /tmp/ficheiro.txt
```

O último comando falha: o ficheiro não existe na nova instância, criada a partir da imagem original e sem qualquer memória do container anterior.

Repita com um volume, que persiste independentemente do ciclo de vida do container:

```
docker volume create dados-icn
docker stop meusite && docker rm meusite
docker run -d --name meusite -p 8080:80 -v dados-icn:/dados icn-site:v1
docker exec meusite sh -c "echo persistente > /dados/ficheiro.txt"
docker stop meusite && docker rm meusite
docker run -d --name meusite -p 8080:80 -v dados-icn:/dados icn-site:v1
docker exec meusite cat /dados/ficheiro.txt
```

O conteúdo sobrevive agora à destruição e recriação do container.

#### Pare e observe

**Observação 5.** Note o que sucedeu em cada um dos dois casos. Relacione o resultado com a distinção, estabelecida na Aula 03, entre o disco de uma máquina virtual e o estado volátil de uma instância.

## 9 Limitar recursos

Execute um container com memória e processador limitados:

```
docker run -d --name limitado --memory=256m --cpus=0.5 nginx:alpine
docker stats --no-stream
```

A coluna MEM USAGE / LIMIT apresenta o limite imposto e não a memória total do computador. O container não consegue exceder o valor declarado: caso o tente, o seu processo é terminado pelo sistema.

Este mecanismo constitui a contrapartida, no modelo de containers, da atribuição de memória a uma máquina virtual. A diferença é relevante: uma máquina virtual reserva a memória declarada, ao passo que um container apenas fica impedido de a exceder, consumindo em cada instante o que efetivamente necessita.

#### Pare e observe

**Observação 6.** Leia a saída de `docker stats --no-stream`. Compare o modelo de limite aqui observado com o modelo de reserva de uma máquina virtual, e indique qual dos dois permite maior densidade de instâncias no mesmo anfitrião.

## 10 Limpeza

Remova os recursos criados:

```
docker stop meusite limitado
docker rm meusite limitado
docker volume rm dados-icn
docker rmi icn-site:v1 icn-site:v2
```

Para remover todos os recursos não utilizados de uma só vez:

```
docker system prune -a
```

### Atenção

O comando `docker system prune -a` remove todas as imagens não associadas a containers em execução, incluindo imagens transferidas noutros contextos. Utilize-o apenas se não tiver trabalho por concluir noutros projetos.

## 11 Resolução de problemas

---

### Cannot connect to the Docker daemon

O serviço não está em execução. Em Windows e macOS, inicie a aplicação Docker Desktop e aguarde o estado `Engine running`. Em Linux, execute `sudo systemctl start docker`.

### permission denied while trying to connect to the Docker daemon socket (Linux)

O utilizador não pertence ao grupo `docker`. Execute `sudo usermod -aG docker $USER`, termine a sessão e volte a iniciá-la. A alteração não produz efeito na sessão em curso.

### port is already allocated

O porto 8080 está ocupado por outro processo. Utilize outro porto no lado do anfitrião, por exemplo `-p 8081:80`, e ajuste o endereço no navegador em conformidade.

### O navegador não apresenta a página

Confirme com `docker ps` que o container se encontra em execução. Caso não apareça na lista, consulte `docker ps -a` e, em seguida, `docker logs <nome>` para identificar a causa da terminação.

### docker build falha com failed to read dockerfile

O comando foi executado fora do diretório que contém o `Dockerfile`, ou o ficheiro foi guardado com extensão. Confirme com `ls` que existe um ficheiro de nome exatamente `Dockerfile`.

### Em Windows, uname não é reconhecido

O comando pertence a sistemas Unix. Utilize `wsl uname -r` em PowerShell.

## 12 Lista de verificação

---

- ☐ Docker instalado e `docker info` executa sem erro.
- ☐ `hello-world` executado, distinção entre imagem e container compreendida.
- ☐ Isolamento observado: `hostname`, processos e sistema de ficheiros próprios (Observação 1).
- ☐ Versões de núcleo comparadas dentro e fora do container, resultado interpretado (Observação 2).
- ☐ Estrutura em camadas de uma imagem inspecionada (Observação 3).
- ☐ Serviço publicado e acessível em `localhost:8080`.
- ☐ Imagem própria construída a partir de `Dockerfile` (Observação 4).

- ☐ Comportamento de persistência com e sem volume verificado (Observação 5).
- ☐ Limites de recursos aplicados e observados (Observação 6).
- ☐ Recursos removidos.

## Continuação

---

A Aula 04 estabelece os mecanismos de núcleo que sustentam o comportamento aqui observado: os *namespaces*, responsáveis pelo isolamento constatado na Observação 1, e os *cgroups*, responsáveis pelos limites da Observação 6. As observações feitas neste laboratório servirão de referência concreta para essa exposição.